EUI OMR ENGINE

Technical Documentation & User Guide

Version: 1.1.1
Author: Mustafa Mahmoud

April 16, 2026

Contents

1 Introduction **2**
1.1 The Problem . . . 2
1.2 Target Users . . . 2

2 System Overview **2**
2.1 High-Level Architecture . . . 2
2.2 Workflow Diagram . . . 3

3 Installation Guide **3**
3.1 Production Deployment (Recommended) . . . 3
3.2 Development Environment Setup . . . 3

4 Usage Guide **4**
4.1 Basic Workflow . . . 4

5 Features **4**
5.1 Smart Calibration (Auto-Tuning) . . . 4
5.2 Probabilistic Peak Detection . . . 4
5.3 Human-in-the-Loop Conflict Resolution . . . 4

6 Architecture Details **5**
6.1 Image Perspective Transformation . . . 5
6.2 Data Models & Concurrency . . . 5

7 Configuration **5**
7.1 `config.json` . . . 5

8 Security Considerations **5**
8.1 Architecture Overview . . . 5

9 Troubleshooting **6**

10 Future Improvements **6**

11 Appendix **6**

Abstract

Educational grading at scale demands precision, speed, and reliability. The EUI Optical Mark Recognition (OMR) Engine resolves the persistent cost and inflexibility of legacy hardware scanners by providing a 100% software-based, computer vision solution. This document comprehensively outlines the architecture, setup, algorithms, and operational usage of the EUI OMR Engine.

1 Introduction

1.1 The Problem

Standardized testing traditionally relies on specialized hardware OMR scanners and proprietary paper. This paradigm is rigid, prohibitively expensive to maintain, and brittle to human error. A faint pencil stroke, a stray smudge, or a skewed paper feed frequently aborts autonomous grading, forcing faculty into tedious manual remarking processes.

1.2 Target Users

- **University Administrators & Operational Staff**: To rapidly grade batches of tests and safely export data to student information systems.
- **Professors & Teaching Assistants**: To customize exam answer keys and resolve ambiguous student entries visually.
- **Developers**: To maintain, extend, and adapt the underlying image processing pipelines or configuration bounds.

2 System Overview

2.1 High-Level Architecture

The system is constructed with a modular, decoupled architecture prioritizing data integrity and asynchronous processing.

- **Presentation Layer (UI)**: Built with PySide6 (Qt framework) enforcing thread-isolated UI updates and a dark-theme, glassmorphism design language.
- **Computer Vision Core**: Python-based image processing utilizing OpenCV (`cv2`) for geometrical transformations and NumPy for high-speed matrix density calculations.
- **Data Ingestion Layer**: Employs PyMuPDF (`fitz`) for vector-to-raster PDF extraction and `openpyxl/pandas` for bidirectional Excel dataset bridging.

2.2 Workflow Diagram

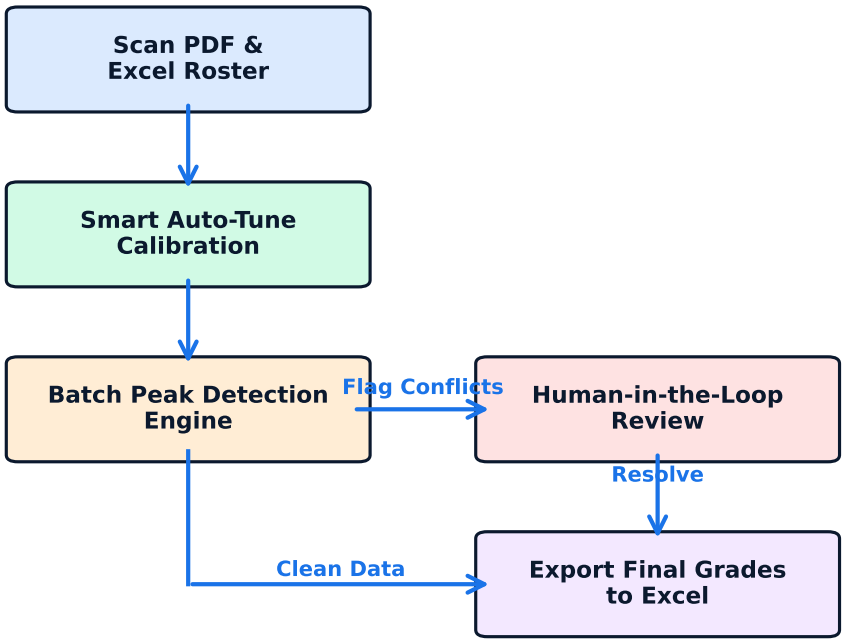

Figure 1: Process Data Flow

3 Installation Guide

3.1 Production Deployment (Recommended)

The EUI OMR Engine is distributed as a portable, standalone executable folder for Windows platforms.

1. Download the latest release (`EUI_OMR_Engine_v1.1.1_Win.zip`).
2. Extract the ZIP folder to your desktop or desired secure drive.
3. Run `EUI_OMR_Engine.exe`. No local Python installation or administrative privileges are required.

3.2 Development Environment Setup

```
# Clone repository
git clone https://github.com/MustafaMahmoud-ILE/eui_omr.git
cd eui_omr

# Setup Virtual Environment
python -m venv .venv
source .venv/Scripts/activate
```

```
# Install Dependencies
pip install -r requirements.txt

# Run Application
python main_entry.py
```

4 Usage Guide

4.1 Basic Workflow

1. **Setup Phase**: Select the destination Excel roster, map the ID/Grade columns, and upload the batch PDF of student scans.
2. **Key Configuration**: Select the correct models (A, B, C, D) for the exam questions via manual UI selection or model scanning.
3. **Initiate Grading**: Click **Start**. The engine silently tunes its sensitivity, processes the batch, and renders the Results Dashboard.
4. **Review & Export**: Correct any red error rows via double-clicking, then export back to Excel.

5 Features

5.1 Smart Calibration (Auto-Tuning)

Desk scanners lack optical consistency. The engine counters this by extracting 5 random, valid pages and systematically stepping through binary threshold coefficients. It identifies the exact density matrix value that distinguishes graphite from paper for the current specific hardware run.

5.2 Probabilistic Peak Detection

The engine measures relative darkness within an array (e.g., all 10 bubbles of an ID column). It identifies the statistical "Peak" of darkness. This allows successful reads of faint marks while ignoring poor erasures or stray pencil noise.

5.3 Human-in-the-Loop Conflict Resolution

No automated system is flawless. When mathematical ambiguity occurs, grading halts for that specific test, and the visual crop is relayed to a GUI modal for operator override.

6 Architecture Details

6.1 Image Perspective Transformation

The core OMR function, `process_array()`, utilizes a 4-point perspective transform. The scanner often feeds paper slightly skewed. The engine hunts for 4 defined geometric corner markers in the image space. Using `cv2.getPerspectiveTransform()`, it aligns the skewed space into a perfectly orthogonal layout. Coordinate `(X, Y)` is then guaranteed to house the correct bubble.

6.2 Data Models & Concurrency

Internal state is passed between threads using immutable Data Classes (e.g., `GradingResult`, `ProjectManager`). The heavy image processing logic is strictly detached into QThread worker classes (e.g., `PDFGraderWorker`, `AutoTuneWorker`) employing Signal/Slot emission to update the UI safely without blocking the main event loop.

7 Configuration

7.1 `config.json`

The Region of Interest (ROI) definitions are exclusively isolated from Python code in an editable `config.json` located parallel to the runtime executable. This allows administrative adjustment of bounding boxes when the physical exam paper template is altered.

```
{
  "student_id": { "x": 1081, "y": 801, "w": 381, "h": 286 },
  "version": { "x": 1081, "y": 1289, "w": 381, "h": 129 },
  "corners": { "tl_x": 84, "tl_y": 74 }
}
```

8 Security Considerations

8.1 Architecture Overview

The EUI OMR Engine processes sensitive educational data falling strictly within local administrative policies.

- **Offline Architecture**: The engine operates 100% client-side. No telemetry, student responses, or grades ever traverse a network or API boundary.
- **Auditability**: Manual overrides are explicitly tracked and logged in the raw data structures, establishing a verification trail if an appeal is filed by a student.

9 Troubleshooting

- **Error: Multiple Pages Skipped**: Ensure the PDF was scanned right-side up or upside down. If the document is fed in *Landscape*, the corner markers will fail to register.
- **Error: Cannot Export to Excel**: Close the target `.xlsx` file in Microsoft Excel. Excel places write-locks on open files, preventing the engine from injecting grades.
- **Error: "Config Not Found" fallback warning**: Ensure `config.json` sits directly adjacent to `EUI_OMR_Engine.exe`.

10 Future Improvements

- **Multi-Processing Processing Pipeline**: While UI concurrency is solved, the grading logic remains CPU-bound on a single core. Implementing `multiprocessing` could chunk PDF loads across all CPU threading pools, significantly accelerating 1,000+ page batches.
- **Visual ROI Configurator**: A graphical tool enabling standard users to draw boxes over a PDF to generate the `config.json` arrays visually rather than typing integer coordinates.
- **Barcode Integration**: Scanning QR or Barcodes for automatic versioning control rather than relying on bubble input.

11 Appendix

For further questions or contributions, access the source repository:
https://github.com/MustafaMahmoud-ILE/eui_omr